Bayes decisions and model evaluation

Confusion matrices

Confusion matrices are a tool to visualize the number of samples predicted as class i and belonging to class j. A confusion matrix is a $K \times K$ matrix whose elements $M_{i,j}$ represent the number of samples belonging to class j that are predicted as class i.

We start considering the Gaussian classifiers (MVG) on the IRIS dataset. We make use of the 2-to-1 dataset split that we employed in previous laboratories.

For the moment we assume that mis-classification costs are all equal to 1. We have seen that, in this case, optimal decisions correspond to predicting the class which has maximum posterior probability. Compute the confusion matrix for the predictions of the MVG classifier on the IRIS dataset. You should get

		Class		
		0	1	2
Prediction	0	19	0	0
	1	0	15	0
	2	0	2	14

If you repeat the same process for the Tied covariance classifier you would get

		Class		
		0	1	2
Prediction	0	19	0	0
	1	0	16	0
	2	0	1	14

Given the limited number of errors, a detailed analysis of the IRIS dataset is not very interesting. We thus turn our attention to a larger evaluation dataset. We consider again a 3-class problem, and a classifier that outputs class-conditional log-likelihoods. For this laboratory, we will consider the classifier and the evaluation data as given, i.e., we will not train a classification model, but we will start from the classifier outputs for a given evaluation set.

The classifier scores for the evaluation set (class-conditional log-likelihoods) and the ground truth labels can be found in files `Data/evalset_ll.npy` and `Data/evalset_labels.npy` , respectively. The score files contains a matrix of log-likelihoods. Each column of the log-likelihood matrix corresponds to a sample, and contains, in its rows, the class-conditional log-likelihoods for the 3 classes. The label file contains an array with the sample ground truth labels. The setup is very similar to the Iris evaluation setup, and, for our purposes, you can consider the provided data is if they were just a larger Iris-like evaluation set.

Given the matrix of class-conditional log-likelihoods, compute the confusion matrix for decisions based on uniform prior and uniform cost assumptions (i.e. maximum posterior class probability decisions). You should get

		Class		
		0	1	2
Prediction	0	210	113	61
	1	137	191	111
	2	53	98	230

Optimal Bayes decisions

We now focus on making optimal decisions when priors and costs are not uniform. Optimal Bayes decisions are decisions that minimize the expected Bayes cost, from the point of view of the recognizer $\mathcal{R}$.

Although in the lectures we denoted classes with $k = 1 \dots K$, in the following we use indices $k = 0 \dots K-1$. This has the advantage that indexing follows the same convention used in numpy matrices, and we have the same representations for binary and multiclass problems.

For a K-class problem, let

$$\boldsymbol{\pi} = \begin{bmatrix} \pi_0 \\ \vdots \\ \pi_{K-1} \end{bmatrix}$$

denote the prior class probabilities. We already discussed how to compute class posterior probabilities:

$$P(C = c|x, \mathcal{R}) = \frac{f_{X|C,\mathcal{R}}(x|c)\pi_c}{\sum_{k=0}^{K-1} f_{X|C,\mathcal{R}}(x|k)\pi_k}$$

We define the cost matrix

$$\boldsymbol{C} = \begin{bmatrix} 0 & C_{0,1} & \dots & C_{0,K-1} \\ C_{1,0} & 0 & \dots & C_{1,K-1} \\ \vdots & \vdots & \ddots & \vdots \\ C_{K-1,0} & C_{K-1,1} & \dots & 0 \end{bmatrix}$$

$C_{i,j}$ represents the cost of predicting class i when the actual class is j.

When we classify a new sample, we do not know its real class, however we can compute the expected Bayes cost of predicting class c according to the posterior class probabilities provided by the recognizer $\mathcal{R}$:

$$\mathcal{C}_{x,\mathcal{R}}(c) = \sum_{k=0}^{K-1} C_{c,k} P(C = k|x, \mathcal{R})$$

The optimal Bayes decision consists in predicting the class c^* which has minimum expected Bayes cost:

$$c^* = \arg\min_c \mathcal{C}_{x,\mathcal{R}}(c)$$

For binary tasks with classes $\mathcal{H}_T = 1$, $\mathcal{H}_F = 0$, we have two costs $C_{0,1} = C_{fn}$ and $C_{1,0} = C_{fp}$, i.e. the costs of false negatives and false positive errors. The expected Bayes cost for predicting either of the two classes are

$$\begin{aligned} \mathcal{C}_{x,\mathcal{R}}(0) &= C_{0,0}P(C = 0|x, \mathcal{R}) + C_{0,1}P(C = 1|x, \mathcal{R}) = C_{fn}P(C = 1|x, \mathcal{R}) \\ \mathcal{C}_{x,\mathcal{R}}(1) &= C_{1,0}P(C = 0|x, \mathcal{R}) + C_{1,1}P(C = 1|x, \mathcal{R}) = C_{fp}P(C = 0|x, \mathcal{R}) \end{aligned}$$

We predict the class c^* that has **minimum** cost $c^* = \arg\min_c \mathcal{C}_{x,\mathcal{R}}(c)$. For the binary task, we can also express c^* as

$$\begin{aligned} c^* &= \begin{cases} 1 & \text{if } \log \frac{C_{fn}P(C=1|x,\mathcal{R})}{C_{fp}P(C=0|x,\mathcal{R})} > 0 \\ 0 & \text{if } \log \frac{C_{fn}P(C=1|x,\mathcal{R})}{C_{fp}P(C=0|x,\mathcal{R})} \leq 0 \end{cases} \\ &= \begin{cases} 1 & \text{if } \log \frac{\pi_1 C_{fn} f_{X|C,\mathcal{R}}(x|1)}{(1-\pi_1)C_{fp} f_{X|C,\mathcal{R}}(x|0)} > 0 \\ 0 & \text{if } \log \frac{\pi_1 C_{fn} f_{X|C,\mathcal{R}}(x|1)}{(1-\pi_1)C_{fp} f_{X|C,\mathcal{R}}(x|0)} \leq 0 \end{cases} \\ &= \begin{cases} 1 & \text{if } \log \frac{f_{X|C,\mathcal{R}}(x|1)}{f_{X|C,\mathcal{R}}(x|0)} > -\log \frac{\pi_1 C_{fn}}{(1-\pi_1)C_{fp}} \\ 0 & \text{if } \log \frac{f_{X|C,\mathcal{R}}(x|1)}{f_{X|C,\mathcal{R}}(x|0)} \leq -\log \frac{\pi_1 C_{fn}}{(1-\pi_1)C_{fp}} \end{cases} \end{aligned}$$

i.e., we can compare the log-likelihood ratio

$$r(x) = \log \frac{f_{X|C,\mathcal{R}}(x|1)}{f_{X|C,\mathcal{R}}(x|0)}$$

with threshold

$$t = -\log \frac{\pi_1 C_{fn}}{(1-\pi_1)C_{fp}}$$

Binary task: optimal decisions

We start considering a 2-class problem extracted from the original 3-class task. In this case, rather than using class-conditional log-likelihood matrices, we represent the classifier outputs as an array of log-likelihood ratios. This array can be loaded from file `Data/evalset_llr_binary.npy` and the corresponding labels are given in `Data/evalset_labels_binary.npy` . Since this is a binary problem, we represent one of the classes as the True (label $\mathcal{H}_T$) and the other class as the False (label $\mathcal{H}_F$) hypothesis.

Write a function that computes optimal Bayes decisions for different priors and costs starting from binary log-likelihood ratios. The function should receive the triplet (π_1, C_{fn}, C_{fp}), corresponding to the cost matrix

$$\boldsymbol{C} = \begin{bmatrix} 0 & C_{fn} \\ C_{fp} & 0 \end{bmatrix}$$

and to prior class probabilities $P(C = \mathcal{H}_\mathcal{T}) = \pi_1, P(C = \mathcal{H}_F) = 1 - \pi_1$.

To verify that your predictions are correct, you can find below the confusion matrices that you should obtain with decisions made with different costs and priors.

T = 1 & F = 0

Prediction \ Class	0	1
0	293	96
1	109	304

$\pi_1 = 0.5$
$C_{fn} = C_{0,1} = 1$
$C_{fp} = C_{1,0} = 1$

When the prior for class 1 increases, the classifier tends to predict class 1 more frequently

Prediction \ Class	0	1
0	271	80
1	131	320

$\pi_1 = 0.8$
$C_{fn} = C_{0,1} = 1$
$C_{fp} = C_{1,0} = 1$

Prediction \ Class	0	1
0	257	75
1	145	325

$\pi_1 = 0.5$
$C_{fn} = C_{0,1} = 10$
$C_{fp} = C_{1,0} = 1$

Prediction \ Class	0	1
0	302	113
1	100	287

$\pi_1 = 0.8$
$C_{fn} = C_{0,1} = 1$
$C_{fp} = C_{1,0} = 10$

We can observe that

- When the prior for class 1 increases, the classifier tends to predict class 1 more frequently
- When the cost of predicting class 0 when the actual class is 1, $C_{0,1}$ increases, the classifier will make more false positive errors and less false negative errors. The opposite is true when $C_{1,0}$ is higher.

Binary task: evaluation

We now turn our attention at evaluating the predictions made by our classifier $\mathcal{R}$ for a target application with prior and costs given by (π_1, C_{fn}, C_{fp}).

At evaluation time, we use the real labels of the test set to evaluate the decisions c^*. As we have seen, we can compute the empirical Bayes risk (or detection cost function, DCF), that represents the cost that we pay due to our decisions c^* for the test data. The risk can be expressed as

$$\begin{aligned} DCF_u = \mathcal{B} &= \sum_{k=0}^{K-1} \frac{\pi_k}{N_k} \sum_{i|c_i=k} \mathcal{C}(c_i^*|k) \\ &= \pi_1 C_{fn} P_{fn} + (1-\pi_1) C_{fp} P_{fp} \end{aligned}$$

where c_i is the actual class for sample i and c_i^* is the predicted class for the same sample. P_{fn} and P_{fp} are the false negative and false positive rates. These can be computed from the confusion matrix $\boldsymbol{M}$. Remember that $\boldsymbol{M}_{i,j}$ represents the number of samples of class j predicted as belonging to class i. We can then compute

$$P_{fn} = \frac{FN}{FN+TP} = \frac{M_{0,1}}{M_{0,1}+M_{1,1}}, \quad P_{fp} = \frac{FP}{FP+TN} = \frac{M_{1,0}}{M_{0,0}+M_{1,0}}$$

Write a function that computes the Bayes risk from the confusion matrix corresponding to the optimal decisions for an application (π_1, C_{fn}, C_{fp}) (use the same values both for computing the cost and for computing the decisions). For the previous four configurations, you should get

(π_1, C_{fn}, C_{fp})	DCF_u $(\mathcal{B})$
(0.5, 1, 1)	0.256
(0.8, 1, 1)	0.225
(0.5, 10, 1)	1.118
(0.8, 1, 10)	0.724

it is un-normalised

The Bayes risk allows us comparing different systems, however it does not tell us what is the benefit of using our recognizer with respect to optimal decisions based on prior information only. We can compute a normalized detection cost, by dividing the Bayes risk by the risk of an optimal system that does not use the test data at all. We have seen that the cost of such system is

$$\mathcal{B}_{dummy} = \min(\pi_1 C_{fn}, (1-\pi_1) C_{fp})$$

Compute the normalized DCF for the aforementioned configurations. You should get

(π_1, C_{fn}, C_{fp})	DCF
(0.5, 1, 1)	0.511
(0.8, 1, 1)	1.126
(0.5, 10, 1)	2.236
(0.8, 1, 10)	0.904

We can observe that only in two cases the DCF is lower than 1, in the remaining cases our system is actually harmful.

Minimum detection costs

We have seen during the lectures that, for binary tasks, we can measure the contribution to the cost due to poor class separation and the contribution due to poor score calibration: our classifier does not produce outputs that represent log-likelihood ratios, thus the theoretical threshold is not optimal anymore (note that this happens even for generative models — for example, the model parameters may not be consistent between training and test population).

Scores can be *re-calibrated* by using a (small) set of labeled samples, that behave similarly to the evaluation population and were not used for model training (i.e. a validation set). Alternatively, we can compute the optimal threshold for a given application on the same validation set, and use such threshold for the test population (K-fold cross validation can be also exploited to extract validation sets from the training data when validation data is not available).

Even if we do not have available such data, it may still be interesting knowing how good the model would perform if we had selected the best possible threshold. To this extent, we can compute the (normalized) DCF over the test set using all possible thresholds, and select its minimum value. This represents a lower bound for the DCF that our system can achieve (minimum DCF in the following).

To compute the minimum cost, consider a set of thresholds corresponding to $(-\infty, s_1 \dots s_M, +\infty)$, where $s_1 \dots s_M$ are the test scores, sorted in increasing order (notice that the DCF can change only when we change a prediction, and that can happen only when the threshold moves "across" one of the evaluation scores). For each threshold t, compute the confusion matrix on the test set itself that would be obtained if scores were thresholded at t, and the corresponding normalized DCF using the code developed in the previous section. The minimum DCF is the minimum of the obtained values.

(π_1, C_{fn}, C_{fp})	min DCF
(0.5, 1, 1)	0.506
(0.8, 1, 1)	0.752
(0.5, 10, 1)	0.842
(0.8, 1, 10)	0.709

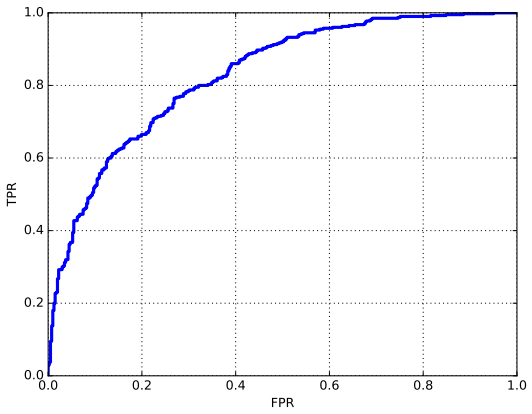
With the except of the first application, we can observe a significant loss due to poor calibration. This loss is even more significant for the two applications which had a normalized DCF larger than 1. In these two scenarios, our classifier is able to provide discriminant scores, but we were not able to employ the scores to make better decisions than those that we would make from the prior alone.

ROC curves

ROC curves are a method to evaluate the trade-off between the different kind of errors for our recognizer. ROC curves can be used, for example, to plot false positive rates versus true positive rates as the threshold varies. Note that these plots do not account for errors due to poor selection of the threshold (i.e. mis-calibration), since they simply plot how the error rates change when we modify the threshold on the evaluation set.

The most commonly used ROC curves plot true positive rates against false positive rates. You can compute true positive rates from false negative rates as $P_{tp} = 1 - P_{fn}$. The ROC curve consists of points $(P_{tp}(t), P_{fp}(t))$ where t is the threshold. By sweeping all possible thresholds we can obtain the coordinates of the ROC curve.

Plot the ROC curve for the 2-class problem scores (array of LLRs). For each threshold, compute the confusion matrix and extract the P_{fn} and P_{fp} as you did in the previous section. Compute P_{tp} as $1-P_{fn}$. Plot the curve that contains, on x-axis, all the false positive rates, and on the y-axis all corresponding true positive rates. You should obtain:



Bayes error plots

The last tool that we consider to assess the performance of our recognizer consists in plotting the normalized costs as a function of an effective prior $\tilde{\pi}$. We have seen that, for binary problems, an application (π_1, C_{fn}, C_{fp}) is indeed equivalent to an application $(\tilde{\pi}, 1, 1)$. We can thus represent different applications with different values of $\tilde{\pi}$. The normalized Bayes error plot allows assessing the performance of the recognizer as we vary the **application**, i.e. as a function of prior log-odds $\tilde{p} = \log \frac{\tilde{\pi}}{1-\tilde{\pi}}$ (note that the prior log-odds are the negative of the theoretical threshold for the considered application).

Compute the Bayes error plot for our recognizer. Consider values of $\tilde{p}$ ranging, for example, from -3 to +3. You can generate linearly spaced values with `effPriorLogOdds = numpy.linspace(-3, 3, 21)` (21 is the number of points we evaluate the DCF at in the example, i.e. the number of values of $\tilde{p}$ we consider). For each value $\tilde{p}$, compute the corresponding effective prior

$$\tilde{\pi} = \frac{1}{1 + e^{-\tilde{p}}}$$

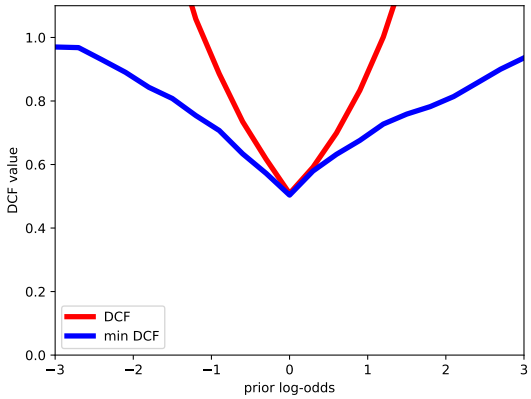
Compute the normalized DCF, and the normalized minimum DCF corresponding to $\tilde{\pi}$. Plot the computed values as a function of log-odds $\tilde{p}$: the x-axis should contain the values of $\tilde{p}$, the y-axis the corresponding DCF. To obtain the DCF, you can re-use the previous code: for each value of $\tilde{\pi}$, compute the DCF for the application $(\tilde{\pi}, 1, 1)$.

You can plot both DCF and minimum DCF over the same figure by using

```
matplotlib.pyplot.plot(effPriorLogOdds, dcf, label='DCF', color='r')
matplotlib.pyplot.plot(effPriorLogOdds, mindcf, label='min DCF', color='b')
matplotlib.pyplot.ylim([0, 1.1])
matplotlib.pyplot.xlim([-3, 3])
```

where `dcf` is the array containing the DCF values, and `mindcf` is the array containing the minimum DCF values.

The Bayes plot should look like

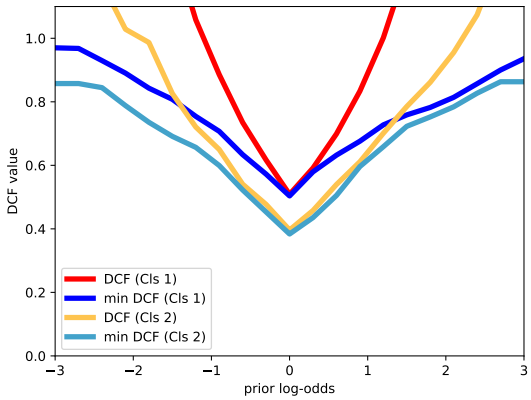


Comparing recognizers

We now employ the tools that we have seen to compare two different recognizers for the same 2-class problem. The two classifiers correspond to different models that compute different LLRs for the same task.
For the first model you can simply use the same LLRs we employed in the previous sections. You can find the LLRs for the second model in `Data/evalset_llr_binary_2.npy`. You should get the following DCFs for the applications we considered:

	Classifier 1		Classifier 2	
(π_1, C_{fn}, C_{fp})	DCF	min DCF	DCF	min DCF
(0.5, 1, 1)	0.511	0.506	0.396	0.386
(0.8, 1, 1)	1.126	0.752	0.748	0.695
(0.5, 10, 1)	2.236	0.842	1.053	0.839
(0.8, 1, 10)	0.904	0.709	0.658	0.604

and the Bayes error plots:



We can conclude that the second model is superior over a wide range of applications (lower DCF), and also produces better calibrated scores (lower gap between DCF and minimum DCF).

Multiclass evaluation [Optional]

We now consider multiclass problems. In contrast with binary problems, the target application cannot be parametrized by a single value. This makes the analysis more difficult, since we do not have a single optimal threshold anymore, but decisions take more complex expressions. It is thus also more difficult to separate the classifier ability to discriminate the classes from the loss due to poorly calibrated conditional likelihoods. For these reasons, we restrict our analysis to confusion matrices and Bayes costs.

Files `Data/evalset_ll.npy` and `Data/evalset_labels.npy` contain class-conditional log-likelihoods and labels for the evaluation population. In this case we have three classes, so the cost matrix has the form

$$\boldsymbol{C} = \begin{bmatrix} 0 & C_{0,1} & C_{0,2} \\ C_{1,0} & 0 & C_{1,2} \\ C_{2,0} & C_{2,1} & 0 \end{bmatrix}$$

and the prior probabilities can be represented by a vector

$$\boldsymbol{\pi} = \begin{bmatrix} \pi_0 \\ \pi_1 \\ \pi_2 \end{bmatrix}$$

with $\pi_1 + \pi_2 + \pi_3 = 1$.

The Bayes cost of classifying a pattern $\boldsymbol{x}_t$ as belonging to class c can be computed as

$$\mathcal{C}_{\boldsymbol{x}_t,\mathcal{R}}(c) = \sum_{k=0}^{K-1} C_{c,k} P(C = k|\boldsymbol{x}_t, \mathcal{R})$$

We can compute the vector of costs

$$\overline{\mathcal{C}}_{\boldsymbol{x}_t,\mathcal{R}} = \begin{bmatrix} \mathcal{C}_{\boldsymbol{x}_t,\mathcal{R}}(0) \\ \mathcal{C}_{\boldsymbol{x}_t,\mathcal{R}}(1) \\ \mathcal{C}_{\boldsymbol{x}_t,\mathcal{R}}(2) \end{bmatrix}$$

from the vector of posterior class probabilities

$$\boldsymbol{P}_{\boldsymbol{x}_t,\mathcal{R}} = \begin{bmatrix} P(C = 0|\boldsymbol{x}_t, \mathcal{R}) \\ P(C = 1|\boldsymbol{x}_t, \mathcal{R}) \\ P(C = 2|\boldsymbol{x}_t, \mathcal{R}) \end{bmatrix}$$

as

$$\overline{\mathcal{C}}_{\boldsymbol{x}_t,\mathcal{R}} = \boldsymbol{C}\boldsymbol{P}_{\boldsymbol{x}_t,\mathcal{R}}$$

We can then predict the class that has the minimum cost.

Compute the optimal class for cost matrix and prior vector

$$\boldsymbol{C} = \begin{bmatrix} 0 & 1 & 2 \\ 1 & 0 & 1 \\ 2 & 1 & 0 \end{bmatrix} \quad , \quad \boldsymbol{\pi} = \begin{bmatrix} 0.3 \\ 0.4 \\ 0.3 \end{bmatrix}$$

$\boldsymbol{C}$ encodes larger errors due to classifying an inferno tercet as belonging to paradiso or vice versa.

Suggestion 1: You can directly work with the matrix of class posterior probabilities for all test samples. Let $\widehat{\boldsymbol{P}}$ be such matrix, the corresponding matrix of costs for all classes and all samples can be computed as $\widehat{\boldsymbol{C}} = \boldsymbol{C}\widehat{\boldsymbol{P}}$. The optimal class can be obtained using `numpy.argmin(..., axis=0)` (note that we take the minimum)

Suggestion 2: You can compute the normalized DCF by computing the Bayes cost of a classifier that always selects the class with minimum prior cost. The cost of classifying a sample as belonging to class c according to the prior probabilities is

$$\mathcal{C}_{\mathcal{P}}(c) = \sum_{k=1}^{K} C_{c,k}\pi_k$$

The cost of a "dummy" system whose decisions are based on the prior alone is the minimum of these costs. In vector form, you can compute the cost of the "dummy" system, i.e. the normalization term required for computing normalized DCF, as `numpy.min(numpy.dot(C, vPrior))`, where `vPrior` is a column vector containing prior probabilities

Suggestion 3: Given optimal class assignments, compute the confusion matrix, and the mis-classification ratios for each class $R_{i,j} = \frac{M_{i,j}}{\sum_i M_{i,j}}$. Once you have the matrix R of mis-classification ratios, you can compute the detection cost by simply computing

$$DCF_u = \mathcal{B} = \sum_{j=1}^{k} \pi_j \sum_{i=1}^{k} R_{i,j} C_{i,j}$$

The normalized cost is obtained by dividing the DCF_u value by the cost of the "dummy" system (see previous suggestion)

You should get

$$\boldsymbol{M} = \begin{bmatrix} 205 & 111 & 56 \\ 145 & 199 & 121 \\ 50 & 92 & 225 \end{bmatrix} \quad , \quad \text{DCF}_u = 0.560 \ , \quad \text{DCF} = \frac{\text{DCF}_u}{0.6} = 0.933$$

i.e., slightly better than using just the prior information system.

A second set of class-conditional log-likelihoods for a second classifier is given in `Data/evalset_ll_2.npy`. With these scores you should get

$$\boldsymbol{M} = \begin{bmatrix} 216 & 77 & 31 \\ 146 & 236 & 143 \\ 38 & 89 & 228 \end{bmatrix} \quad , \quad \text{DCF}_u = 0.485 \ , \quad \text{DCF} = \frac{\text{DCF}_u}{0.6} = 0.808$$

For an application with uniform class priors and costs, $\boldsymbol{\pi}_k = \frac{1}{3}$, $C_{i,j} = 1 \ \forall i \neq j$, you should obtain

	DCF_u	DCF
Classifier 1	0.476	0.714
Classifier 2	0.415	0.623

Project

Analyze the performance of the MVG classifier and its variants for different applications.

Start considering five applications, given by (π_1, C_{fn}, C_{fp}):

- $(0.5, 1.0, 1.0)$, i.e., uniform prior and costs
- $(0.9, 1.0, 1.0)$, i.e., the prior probability of a genuine sample is higher (in our application, most users are legit)
- $(0.1, 1.0, 1.0)$, i.e., the prior probability of a fake sample is higher (in our application, most users are impostors)
- $(0.5, 1.0, 9.0)$, i.e., the prior is uniform (same probability of a legit and fake sample), but the cost of accepting a fake image is larger (granting access to an impostor has a higher cost than labeling as impostor a legit user - we aim for strong security)
- $(0.5, 9.0, 1.0)$, i.e., the prior is uniform (same probability of a legit and fake sample), but the cost of rejecting a legit image is larger (granting access to an impostor has a lower cost than labeling a legit user as impostor - we aim for ease of use for legit users)

Represent the applications in terms of effective prior. What do you obtain? Observe how the costs of mis-classifications are reflected in the prior: stronger security (higher false positive cost) corresponds to an equivalent lower prior probability of a legit user.

We now focus on the three applications, represented in terms of effective priors (i.e., with costs of errors equal to 1) given by $\tilde{\pi} = 0.1$, $\tilde{\pi} = 0.5$ and $\tilde{\pi} = 0.9$, respectively.

For these three applications, compute the optimal Bayes decisions for the validation set for the MVG models and its variants (to limit the number of experiments, do not consider any pre-processing — in real tasks, however, it may be worth analyzing also the effects of pre-processing strategies like, for example, PCA). Compute actual DCF and minimum DCF for the different models. Compare the models in terms of minimum DCF. Which models perform best? Are the relative results of the models consistent over the different applications? Now consider also actual DCFs. Are the models well calibrated (i.e., with a calibration loss in the range of few percents of the minimum DCF value) *for the given applications*? Are there models that are better calibrated than others for the different applications?

Compute the Bayes error plots for the MVG, Tied and Naive Bayes Gaussian classifiers. For each model plot both minimum and actual DCF. Consider prior log odds in the range $(-4, +4)$. What do you observe? Are model rankings consistent across applications (minimum DCF)? Are models well-calibrated over the considered range of applications?